

EC5209 — Advanced Computer Networking
OOP with C++ — Spring 2026

Factory Simulation

Project Requirements & Deliverables

Fill this before May 9. With team members and project name
https://docs.google.com/spreadsheets/d/1rnwR6T955_a2eEn9HT3mliE5K86nt3sCKHx8GP2oqJk/edit?usp=sharing

1. Project Overview

In this project you will design and implement a **factory simulation** in C++, controlled through an interactive GUI built with **Dear ImGui**. By completing it, you will demonstrate mastery of the four pillars of object-oriented programming (abstraction, encapsulation, inheritance, polymorphism) in a non-trivial, interactive system.

Learning objectives

- Design a type hierarchy such that the simulation loop can operate on every factory object without branching on its concrete type — your design must make this possible, and your UML must justify why.
- Ensure that adding a new machine type to the factory requires zero changes to the simulation loop or the UI rendering loop — if either loop must be modified, the design is incomplete.
- Hide the internal state of every object so that no other part of the system can corrupt it directly — a code reviewer must not be able to find a public data member on any simulation class.
- Create a clean separation of UI from backend. How you pass data between the two sides is your design decision.
- Collaborate as a team. Use Github to publish your project.

2. Factory Scenario

Your factory is your design. Choose any manufacturing domain you like — a bakery, a car plant, a semiconductor fab, a furniture workshop, or a generic factory— as long as it satisfies the structural requirements below. The pipeline table shows one example (a parts factory) to illustrate what a valid pipeline looks like; you are not required to follow it.

Stage	Description
-------	-------------

Raw material input (example)	Arrives every N ticks; joins Cutter queue.
Cutter-01	Cuts raw material into parts. Process time: (for ex: 3 ticks)
Conveyor C1	Transfers parts to Assembler. Capacity: (for ex: 5 items)
Assembler-01	Assembles parts into sub-assemblies. Process time: (for ex: 5 ticks)
Conveyor C2	Transfers sub-assemblies to Painter. Capacity: (for ex: 5 items)
Painter-01	Paints and finishes the product. Process time: (for ex: 4 ticks)
Finished goods	Output counter increments; product leaves the system.

Note: If Conveyor logic is hard to implement for you, **you can skip the Conveyor** in your project. Or implement it in second phase, after completing the first phase with only Machines.

2.1 Required Simulation Scenarios

Your simulation must support few among these four of scenarios, selectable at runtime from a dropdown in the ImGui UI: The Key OOP concepts shown in the table are not mandatory, it's just a suggestion.

Scenario	Description	Suggested Key OOP concept
Normal flow	Balanced pipeline; all machines run at default speed.	Polymorphic update()
Bottleneck	Assembler process time increases to 12 ticks, backing up C1.	Composition; queue capacity logic
Random breakdowns	Machine breakdown probability raised to 6%; technician dispatched automatically.	State machine; Worker subclass
Overflow	Slow assembler causes conveyor queue overflow; products lost and logged.	Encapsulation; event logging

Note: if Conveyor logic is hard to implement, **just implement two scenarios will also be enough**
 1) Normal flow 2) Random Breakdowns.

3. Sample Class Hierarchy

Here is a sample class hierarchy listed below. You are free to change and modify this class hierarchy entirely based on your design decisions. Your UML diagram should reflect your class hierarchy.

One abstract root class	All simulation objects inherit from a single base. The main loop stores them as base pointers and calls their methods without a single if/else on the type.
Two levels of inheritance	At least one intermediate abstract class between the root and the concrete classes. Example: root → abstract machine → Oven, Mixer, Packager.
Two or more concrete machine types	Each machine type has its own process time and its own breakdown probability. They are interchangeable in the simulation loop.
Two product stages	An abstract product base class with at least two concrete subclasses representing the start and end of the pipeline.
No public data members	Every field on every class is private or protected. State is read or changed only through methods.

3.1 Sample Interfaces

Some sample key interfaces in the abstract machine class could be :

Method signature	Contract
virtual void update(int tick) = 0	Advance internal state by one tick. Must not call any ImGui functions.
virtual std::string getInfo() const = 0	Return a short human-readable summary used in the event log.

3.2 Sample ER Diagram Components

The ER diagram can have these objects.

- SimulationObject ↔ Machine (IS-A)
- Machine ↔ Conveyor (Machine feeds into Conveyor — one-to-one)
- Conveyor ↔ Product (Conveyor holds many Products — one-to-many)
- Factory (aggregate) ↔ all SimulationObjects (Factory owns many — composition)

4. Sample ImGui UI Requirements

Your application must display the following ImGui windows. Additional windows are welcome,

Window	Minimum contents
Simulation Control	Start / Pause / Reset buttons; speed slider (1×–5×); scenario selector dropdown; live tick counter.
Factory Floor	Visual map of machines and conveyors(optional). Machines color-coded by state. Conveyor load shown via ImGui::ProgressBar.
Inspector	Displays state, health bar, queue depth, output count, and process time for the currently selected machine. Includes Force Break and Instant Repair buttons.
Event Log	Scrollable timestamped list of events (machine state changes, overflow, finished goods). Clear button required.
Statistics	Running totals: finished goods, WIP count, total breakdowns, lost products.

ImGui widget checklist (minimum required)

ImGui::Button — Start, Pause, Reset, Force Break, Instant Repair, Clear Log
 ImGui::SliderInt — simulation speed
 ImGui::Combo or ImGui::ListBox — scenario selector
 ImGui::ProgressBar — machine health, machine progress, conveyor load
 ImGui::TextColored — color-coded machine state labels
 ImGui::BeginChild / EndChild — scrollable event log
 ImGui::Selectable — clickable machine entries in the floor overview

4.1 Decouple UI and Backend

In your design, decouple the UI from backend. While designing there should be only one file (for ex: main.cpp) that should see both sides. Example work flow is explained here

Step 1 — UI layer. `ImGui::Button("Start")` returns `true` for exactly one frame. The only thing `renderInspector()` does with that is `set cmd.startWork = true`. It never touches the `Machine` object.

Step 2 — Bridge. `MachineCmd` is just a plain struct sitting in memory — a carrier pigeon. It has no methods, no logic, no idea what `ImGui` or `Machine` are.

Step 3 — main.cpp. This is the only file that sees both sides. It calls `factory.applyCmd(cmd)`, then immediately clears `cmd = {}` so the command is not applied twice next frame.

Step 4 — Backend. `Machine::applyCmd()` reads the flag, checks that the current state is `IDLE`, and flips `state = WORKING`. No `ImGui` header is anywhere near this file.

Step 5 — Return trip. `snapshot()` copies the new `state` into a fresh `MachineSnap` struct. Next frame, the UI reads `snap.state`, sees `WORKING`, and turns the badge green. The UI is always one frame behind — it never reads `machine.state` directly.

The key insight for your students: **the button click never reaches the backend directly.** It leaves a note (`cmd.startWork = true`), and `main.cpp` delivers that note on the backend's terms, at the right moment in the loop.

5. Project Timeline

The project runs from **Week 1 (May 5)** to **Week 5 (June 6)**. There are three milestone check-ins and one final submission. All deadlines are 23:59 KST on the stated date.

Week	Dates	Milestone	Expected deliverable
1	May 5–9	Kick-off	Fill the excel sheet https://docs.google.com/spreadsheets/d/1rnwR6T955_a2eEn9HT3mliE5K86nt3sCKHx8GP2oqJk/edit?usp=sharing
3	May 19–23	M1 — Core logic	I will check the project Progress. You should be ready with UML and basic code structure with the UI elements.
5	Jun 2–6	Final submission	All deliverables submitted (see Section 5). GitHub Pages live. by June 6, 23:59 KST.

5. Deliverables

5.1 GitHub Repository

Submit the URL of your team's GitHub repository. The repository must contain:

- `src/` — all C++ source files and headers
- `CMakeLists.txt` / `Tasks.json` / `Makefile` for `ImGui`
- UML diagram (ER Diagram) – Hand drawn diagram will be fine.

9. Academic Integrity

You may use Dear ImGui documentation, online C++ references, and AI coding assistants for **understanding concepts** and **debugging**. You may not submit code written entirely by an AI assistant, copy another team's repository, or misrepresent individual contributions in the commit history.

All team members must have meaningful commits. During the final presentation week, any team member may be asked to explain any part of the codebase.